

UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA

FACULTAD DE INGENIERÍA

MANUAL TÉCNICO DE DESARROLLO E INFRAESTRUCTURA

Sistema de Gestión de Recursos del Centro de Cómputo (SGRC)

- **Programa Educativo:** Ingeniería en Ciencias de la Computación
- **Asignatura:** Bases de Datos & Desarrollo Basado en Plataformas
- **Grupo:** 6cc2
- **Equipo de Desarrollo:**
 - Nicolás Paúl Nevárez Loera (Matrícula: 367886)
 - Jonathan Gandara Salazar (Matrícula: 374357)
 - Samuel Eduardo García Gómez (Matrícula: 367651)
- **Fecha y Lugar:** Chihuahua, Chih. 26 de Mayo 2026

1. Sistema/Aplicación

a. Nombre del sistema

Sistema de Gestión de Recursos del Centro de Cómputo (SGRC).

b. Descripción y delimitación del sistema

El SGRC es una aplicación web full-stack diseñada para digitalizar y automatizar los procesos administrativos del Centro de Cómputo en la Facultad de Ingeniería de la UACH. Su alcance se delimita a la gestión de tres recursos principales: reservación y control de acceso a 6 cubículos de estudio físico, administración del inventario de equipamiento tecnológico prestable (con generación de vales en PDF), y un módulo de vinculación académica para tutorías extraescolares. El sistema opera dentro de la red institucional, sirviendo a alumnos, docentes y personal administrativo.

c. Objetivo general

Proveer una plataforma centralizada, concurrente y auditable que erradique los empalmes de horarios y el uso de bitácoras físicas, garantizando un acceso equitativo y transparente a las instalaciones, herramientas y asesorías de la facultad.

d. Objetivos específicos

- Automatizar el proceso de apartados de cubículos previniendo colisiones de horario en tiempo real.
- Implementar un control de asistencia estricto (tolerancia de 15 minutos) mediante códigos QR dinámicos.
- Aplicar bloqueos automáticos en las cuentas de usuarios que incurran en inasistencias o cancelaciones tardías.
- Digitalizar el control de inventario de equipos y generar comprobantes PDF para entrega en ventanilla.
- Proporcionar un catálogo virtual donde los docentes oferten tutorías y los alumnos puedan agendarlas.

e. Descripción de tipos de usuarios

- **ADMIN:** Personal administrativo. Acceso total para gestionar cubículos, aprobar préstamos, auditar sanciones y gestionar inventario.
- **STUDENT:** Alumno de la facultad. Puede reservar cubículos (límite de 2 horas), pedir equipo y agendar tutorías. Es vulnerable a sanciones algorítmicas del sistema.
- **TEACHER:** Docente. Dispone de los mismos privilegios del estudiante pero de forma ilimitada, es inmune a las penalizaciones automatizadas y puede publicar bloques de tutoría.
- **RECEPTOR:** Usuario de sistema asignado a los iPads empotrados en los cubículos. Su interfaz es de solo lectura, bloqueada para mostrar únicamente el código QR de check-in.

f. Entorno operativo del sistema

El sistema se despliega en un entorno virtualizado utilizando Docker Compose. La persistencia se aloja en un contenedor MySQL 8.0 y la mensajería en un contenedor RabbitMQ. El backend Spring Boot 3 (Java 21) se ejecuta en el puerto 3000, mientras que el frontend React 18 opera a través de Vite en el puerto 5173, utilizando un proxy inverso para la comunicación con las APIs. Diseñado para consumirse a través de navegadores web modernos (Chrome, Firefox, Safari) en dispositivos móviles y de escritorio.

2. Especificación de Requerimientos

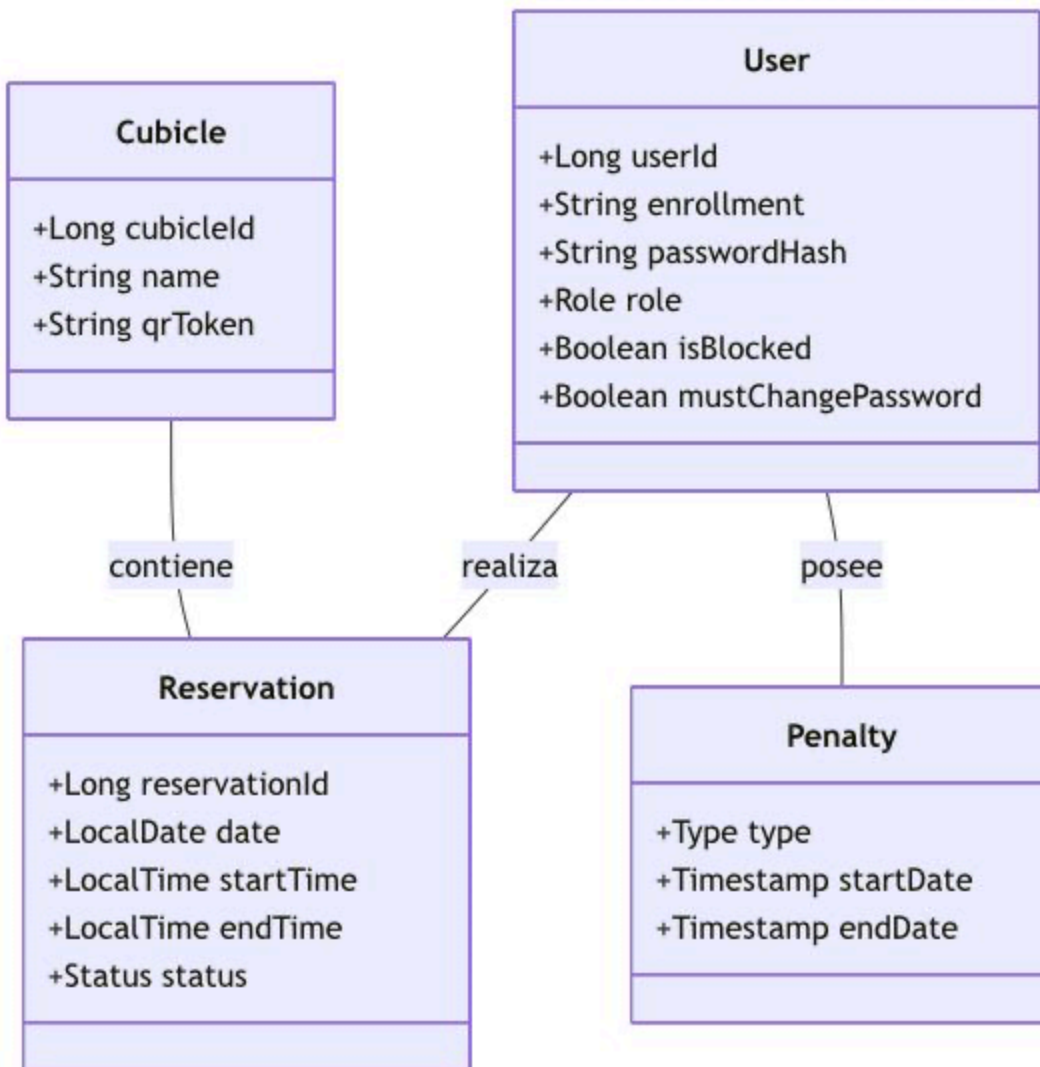
a. Requerimientos Funcionales

- **RF01:** El sistema debe autenticar usuarios mediante credenciales (matrícula y contraseña) emitiendo un JWT.
- **RF02:** El sistema debe exigir el cambio de contraseña genérica en el primer inicio de sesión.
- **RF03:** El sistema debe permitir a estudiantes reservar cubículos por un máximo de 2 horas continuas sin colisiones.
- **RF04:** El sistema debe cancelar automáticamente las reservas no reclamadas (check-in) al minuto 16 y penalizar al usuario con un bloqueo de 24 horas.
- **RF05:** El sistema debe permitir la validación de asistencia escaneando un código QR desde la cuenta del usuario.
- **RF06:** El sistema debe generar un vale en formato PDF al aprobar una solicitud de equipo prestable.
- **RF07:** El sistema debe permitir a docentes publicar materias y horarios de tutoría para los estudiantes.

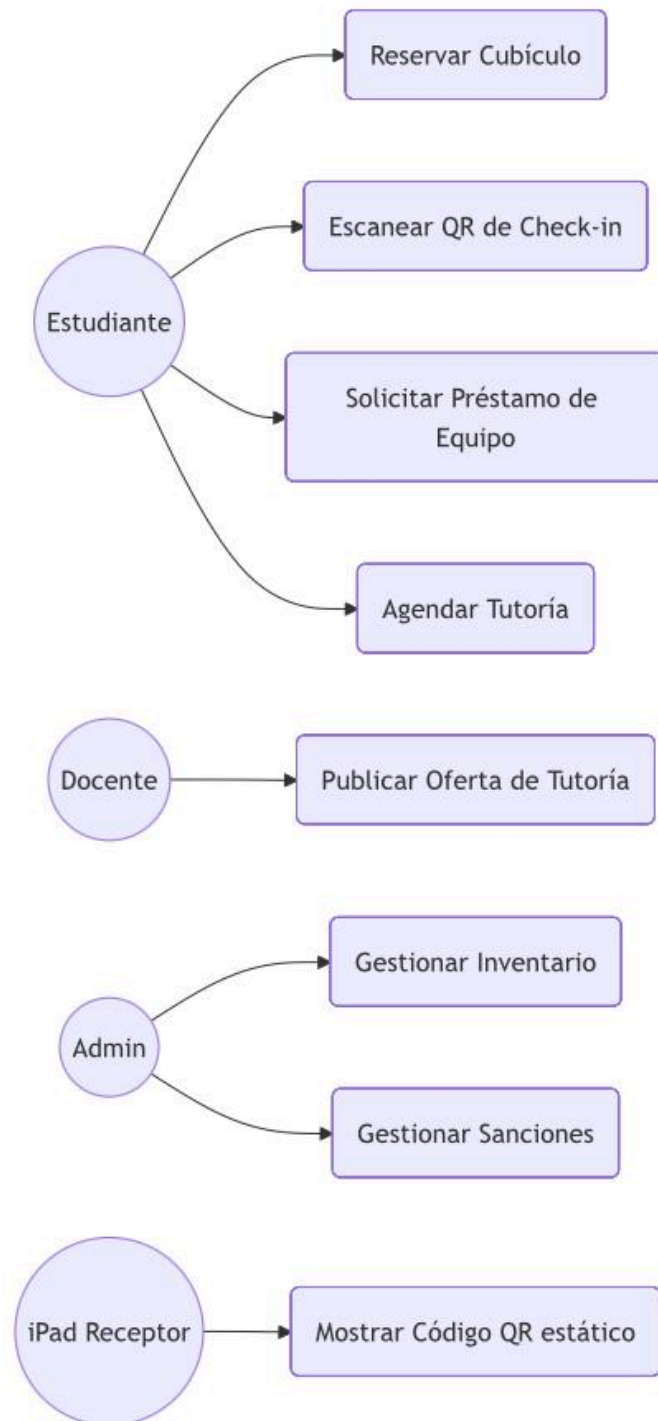
b. Modelado del Sistema

i. Diagrama de Clases

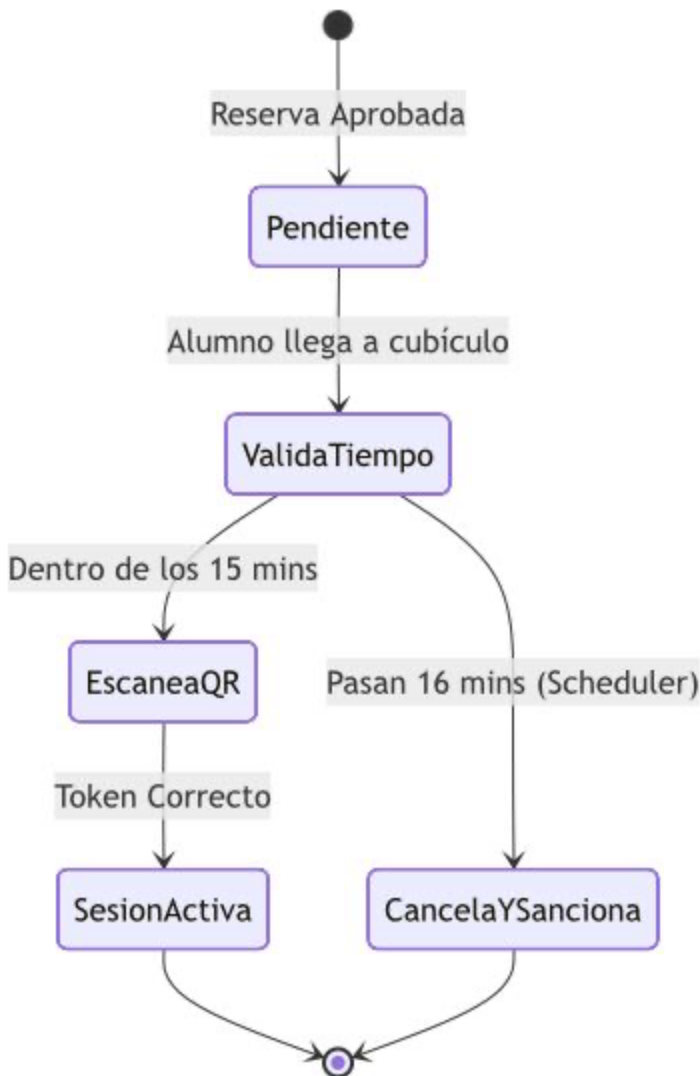
(Representación de las principales entidades de dominio y sus relaciones semánticas en la capa de negocio JPA).



ii. Diagrama de Casos de Uso



iii. Diagrama de Actividades (Flujo de Check-in)



c. Requerimientos No Funcionales

- **RNF01 (Seguridad):** Las contraseñas deben almacenarse encriptadas mediante BCrypt. Las rutas de API deben estar protegidas por JWT.
- **RNF02 (Rendimiento):** El sistema utilizará un Scheduler dinámico asíncrono para las cancelaciones, evitando cuellos de botella por escaneos cronometrados masivos en BD.
- **RNF03 (Interfaz):** La UI debe ser 100% responsiva, priorizando el uso en dispositivos móviles para facilitar el escaneo en sitio.
- **RNF04 (Tiempo Real):** Las actualizaciones críticas de sesión se realizarán vía WebSockets (STOMP).

3. Arquitectura del Sistema/Aplicación

a. Layer/Tier's

La solución implementa una arquitectura N-Tier (3 capas lógicas) estrictamente separadas: Capa de Presentación (Single Page Application delegando el renderizado al cliente), Capa de Lógica/Aplicación (Servidor REST y Broker de WebSockets sin estado), y Capa de Datos (Base de datos relacional y broker de eventos AMQP).

b. Frontend/Backend

- **Frontend:** Desarrollado con React 18 y empaquetado por Vite. Utiliza Tailwind CSS para los estilos de la interfaz y un gestor de estados nativo basado en Hooks (useState, useEffect).
- **Backend:** Construido con Spring Boot 3 y Java 21. Expone controladores REST consumibles vía JSON. Implementa Spring Security para la validación perimetral y Spring Data JPA (Hibernate) para el mapeo relacional.

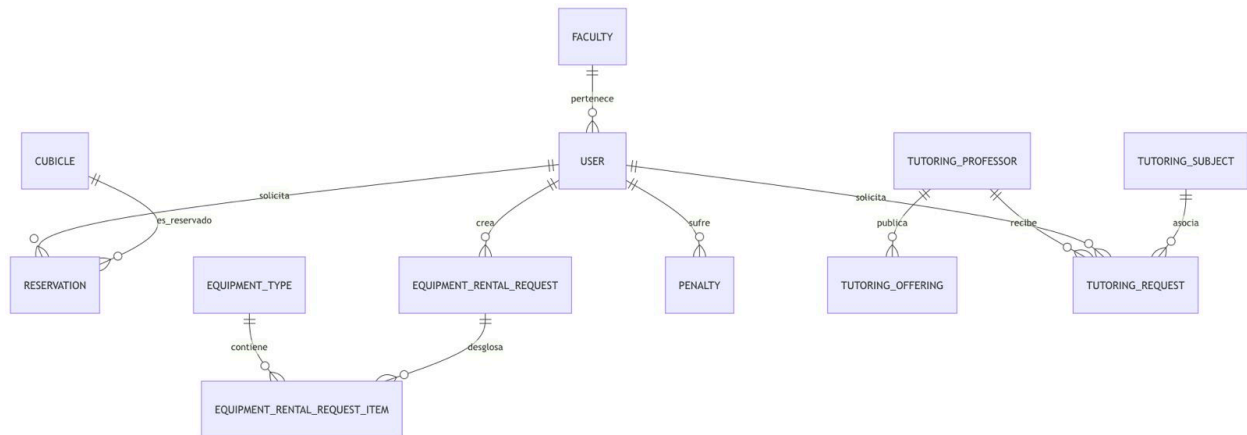
c. Estructura modular del sistema

El sistema se organiza en tres grandes módulos transaccionales desacoplados:

1. **Módulo de Reservas y Sanciones:** Orquesta los empalmes, la agenda, la validación del QR y gestiona el motor de reglas de castigo (Bloqueos por NO_SHOW o LATE_CANCELLATION).
2. **Módulo de Préstamos:** Maneja el catálogo equipment_type, registra el carrito de ítems solicitados y renderiza los PDF's de validación documental.
3. **Módulo de Tutorías:** Un subsistema aislado que interconecta profesores y alumnos, manejando asignaturas y un catálogo de disponibilidad horaria.

4. Base de Datos

a. Diagrama Conceptual (Entidad/Relación)



b. Esquema Lógico de la Base de Datos

i. Especificación Tablas (Normalizadas hasta BCNF)

El esquema está diseñado para cumplir la Forma Normal de Boyce-Codd (BCNF), eliminando redundancias operativas y datos fantasma tras la depuración de tablas heredadas. El modelo operativo incluye 9 tablas activas:

- faculty: Catálogo institucional base (ID, nombre).
- user: Registro universal de usuarios (ID, matrícula, hashes bcrypt, role, flags de estado).
- cubicle: Entidades espaciales (ID, nombre, qr_token único).
- reservation: Control transaccional de tiempos (user_id, cubicle_id, fechas, horas, status).
- penalty: Histórico de bloqueos (user_id, admin_id, tipo de infracción, fechas de activación).
- equipment_type: Maestro de inventarios (stock total, stock disponible, descripción).
- equipment_rental_request / item: Cabecera y detalle (1:N) de las órdenes de equipo.
- tutoring_professor, _subject, _offering, _request: Sub-modelo relacional (N:M resuelta) para emparejar disponibilidad académica.

ii. Integridad de Datos (Constraints)

La integridad a nivel de base de datos se blindo mediante las siguientes reglas:

- **Primary Keys (PK):** AUTO_INCREMENT en todas las tablas transaccionales.
- **Foreign Keys (FK):** Reglas restrictivas ON DELETE RESTRICT para impedir la corrupción o el borrado de usuarios/cubículos que posean historial de reservas activo.
- **Unique Keys (UK):** Exclusividad para correos (email), matrículas (enrollment) y el

identificador físico del cubículo (qr_token).

- **Check Constraints:** Validación matemática a nivel SQL (CHECK (end_time > start_time)) y protección de stocks (CHECK (available_stock <= total_stock)).

5. Lógica / Reglas del Negocio

El corazón algorítmico del SGRC está sujeto a restricciones inquebrantables de comportamiento:

- **Tolerancia de 15 Minutos:** Ningún usuario puede ingresar si su llegada física (escaneo QR) ocurre en un tiempo > start_time + 15 min.
- **Auto-Sanción de Servidor:** A nivel backend, un Task Scheduler por reservación transiciona el estatus a CANCELLED y levanta la bandera is_blocked = 1 en la cuenta del alumno por un lapso de 24 horas.
- **Regla de Capacidades:** El estudiante está topado a solicitar cubículos por un periodo máximo de 2 horas de duración continua. Esta regla no aplica para usuarios con rol TEACHER.
- **Control de Empalmes:** Una reserva será rechazada por HTTP 409 (Conflict) si (hora_solicitada_inicio < existente_fin) AND (hora_solicitada_fin > existente_inicio) para el mismo espacio.
- **Bloqueo Recursivo:** Si is_blocked = 1, el API intercepta y rechaza cualquier solicitud POST hacia reservaciones o préstamos con un código HTTP 403.

6. Descripción Interfaz de la aplicación

La Interfaz Gráfica (UI) está diseñada como una Single Page Application (SPA). Su diseño es minimalista, adoptando las tipografías corporativas UACH (Praxis y Alverata). Se priorizó un diseño *Mobile-First* utilizando la grilla de utilidades de Tailwind CSS, considerando que el flujo crítico del sistema (reservar y escanear el QR) ocurre en los pasillos de la facultad desde dispositivos móviles. Las pantallas principales se dividen por contexto y módulos en el Dashboard. El rol RECEPTOR cuenta con una interfaz exclusiva y congelada que no presenta barra de navegación ni menús, renderizando el código QR asociado al cubículo a pantalla completa para evitar su manipulación por parte de los transeúntes.

7. Descripción de reglas de seguridad (Acceso / Operación)

La protección de la infraestructura se soporta sobre la librería Spring Security:

- **Autenticación Apátrida:** No se utilizan sesiones de servidor JSESSION. La identidad viaja empaquetada y firmada digitalmente en un token JWT (Bearer Token) que contiene el ID, rol y la bandera `must_change_password`.
- **Hashes de Contraseña:** El servidor aplica un cifrado unidireccional utilizando el algoritmo BCrypt con *salt* dinámico para el almacenamiento en MySQL.
- **Role-Based Access Control (RBAC):** Los endpoints del backend exigen anotaciones perimetrales (ej. `@PreAuthorize("hasRole('ADMIN')")`). Cualquier escalamiento de privilegios desde el cliente es rechazado automáticamente.
- **Requisito de Primer Ingreso:** El sistema evalúa la bandera `must_change_password` para forzar al alumno a configurar una clave privada antes de exponer las pantallas operativas en React.

8. Conclusión

El desarrollo e implementación del Sistema de Gestión de Recursos del Centro de Cómputo (SGRC) v1.5 representa una solución arquitectónica madura y de grado de ingeniería para la Facultad de Ingeniería de la UACH. La aplicación de prácticas como la separación de responsabilidades (3-Tier), la normalización relacional (BCNF), la automatización mediante procesos programados (Schedulers) y la seguridad basada en JWT, aseguran que la plataforma ofrezca un entorno escalable, eficiente y blindado para administrar el patrimonio e instalaciones estudiantiles.

9. Anexos

Deuda Técnica y Observaciones a Futuro:

Para la evolución futura del sistema o despliegues en producción extendida, se documentan los siguientes compromisos técnicos actuales:

1. **Rutas Protegidas en UI:** React Router carece de "Guards" del lado del cliente; el ruteo se frena por la falta de token, pero depende de la respuesta 403 del servidor.
2. **Refresh Tokens:** El JWT tiene un tiempo de vida duro de 24h. No existe un flujo de renovación automática (Refresh Token), lo que requiere re-autenticación manual al expirar.

3. **Gestión del Esquema DDL:** El sistema arranca con ddl-auto update en el motor JPA. En entornos productivos de alta sensibilidad, este parámetro debe transicionar a validate para evitar mutaciones accidentales del esquema.
4. **Secretos en Texto Plano:** La llave criptográfica (Secret Key) utilizada para firmar los JWT se almacena directamente en el archivo application.properties. Deberá migrarse hacia una variable de entorno inyectada desde Docker.